But when an entity is reloaded into memory, its state is not restored. In the container example, you get the container back, but it's empty, with no cargo. So, you probably need something more than sharding. You need Akka Persistence.

9.2 Persistence

Akka Persistence follows the event sourcing (ES) modeling style. This means events that take place in your actor are attached like a ledger rather than being kept as a single value that is updated and replaced after each event.

DEFINITION An *event* is the output (stored as a ledger entry) that a persistent actor generates when it receives a message.

This ledger is called a *journal* and is a table in a database that only grows (appends). The entire history of a persistent actor is stored in its journal. The journal begins with a message: a command representing the intention to change the actor. But before this transformation can take place, the command must be validated by executing it in the actor to ensure that the new state created by the command is accepted. If it is, an event can be generated and stored, as shown in figure 9.5.

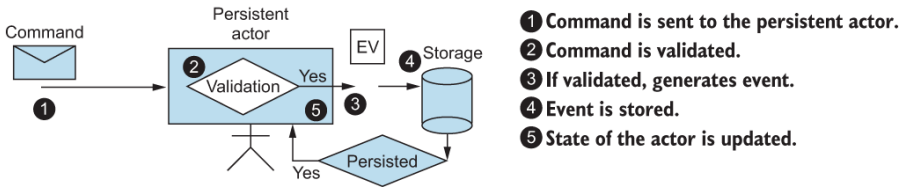


Figure 9.5 From command to a stored event

When the actor needs to be restarted, it reads the journal and replays all the events. Thus, the actor returns to its last known state, as illustrated in figure 9.6. Let's look at this in action.

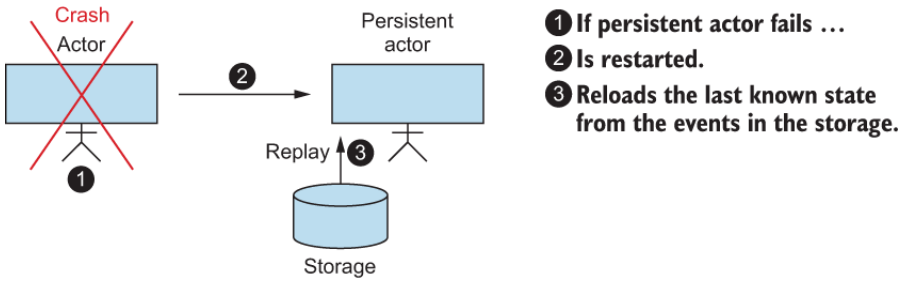


Figure 9.6 Persistent actor recovery

9.2.1 The ingredients